DBeaver

Structured Query Language 101

Hands-on with DBeaver · Texas Linux Festival 2026

Speaker

David Stokes

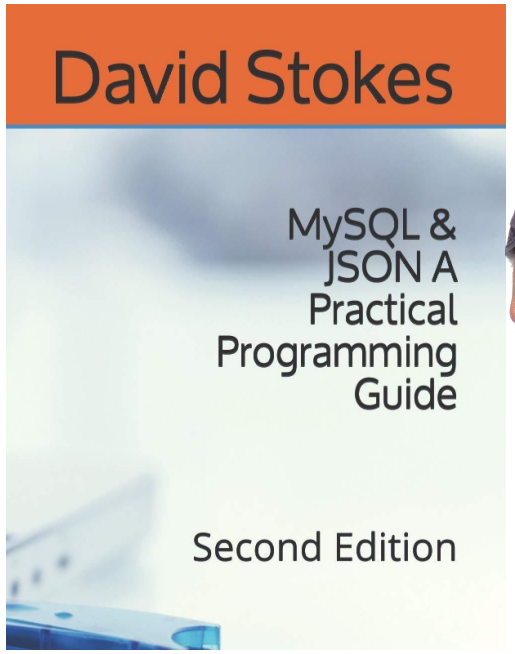
https://github.com/davestokes/sql101

ABOUT ME

Dave Stokes

Community Manager, DBeaver
David.Stokes@DBeaver.com
@Stoker

- MySQL community since 2007 (MySQL AB)
- Author, MySQL & JSON: A Practical Programming Guide
- Has explained JOINs on more napkins than anyone should





LEARNING OBJECTIVES

By the end of today you will be able to…

Read data

Write SELECT queries that pick columns, sort and limit results.

Find the right rows

Narrow results with WHERE, AND/OR, IN, BETWEEN, LIKE and NULL checks.

Change data safely

INSERT, UPDATE and DELETE, with transactions as your safety net.

Summarize

Count, total and average with GROUP BY and HAVING.

Combine tables

Use INNER and LEFT JOIN to answer real questions.

Drive DBeaver

Navigate, run, format and export with DBeaver Community Edition.

SIMPLER One free tool, one sample database, and SQL that carries over to MySQL, PostgreSQL and more.

AGENDA

Today's plan: three hours, seven parts

0:00	Welcome and setup check
0:10	Part 1 · Databases, SQL and DBeaver
0:25	Part 2 · SELECT: reading data
0:50	Part 3 · WHERE: finding the right rows
1:15	Part 4 · INSERT, UPDATE, DELETE
1:35	**Break (10 minutes)**
1:45	Part 5 · Built-in functions
2:00	Part 6 · Grouping and aggregates
2:25	Part 7 · JOINs
2:45	Putting it together, Q&A

SIMPLER Every part ends with a hands-on exercise, and the answers are on the very next slide.

SETUP CHECK

Get DBeaver and the sample database running

1

Install DBeaver Community Edition

Free and open source (Apache 2.0). Get it from dbeaver.io/download for Linux, macOS or Windows.

2

Create the sample database

Say yes when DBeaver offers it on first launch, or use Help › Create Sample Database. It is a small SQLite file: no server needed.

3

Open it

In Database Navigator, expand DBeaver Sample Database (SQLite), then Tables. You should see Album, Artist, Customer, Track and friends.

COMMUNITY EDITION

Stuck?

Raise a hand, or pair up with a neighbour: one laptop is plenty for two people.

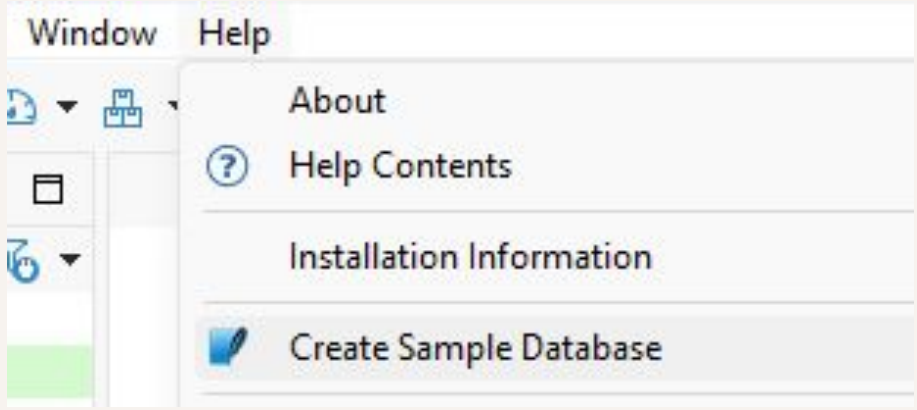
The sample database is SQLite, so nothing to configure, no passwords, no ports.

Everything we break today can be recreated in seconds.

SIMPLER A local SQLite file means everyone gets the same data in two clicks, on any operating system.

RESET BUTTON

Use this if you ever need to recreate the sample database



1 Delete the old connection

Right-click DBeaver Sample Database (SQLite) in Database Navigator › Delete.

2 Help › Create Sample Database

DBeaver copies a fresh Chinook.db into your workspace and reconnects.

3 Back to work

Every table and row is back to the original, ready for the next exercise.

Tip If the connection still exists, DBeaver only says the sample database already exists.

SIMPLER Made a mess? Two clicks and you have a brand-new sample database, so experiment freely.

HOW WE WILL WORK

Type along, break things, ask questions

Type the examples

Retyping beats copy-paste for learning. Small typos teach you to read error messages.

Try the exercises

Each part ends with 2-3 tasks. Answers follow so nobody stays stuck.

Break things

It is a sample database. If it gets messy, delete it and create it again.

TRY IT NOW

```
-- Your first query.
-- Click in the line, press Ctrl+Enter
-- (Cmd+Enter on macOS)
SELECT 'Hello, Texas!' AS greeting;
```

If you see one row with Hello, Texas! in the results panel, you are ready.

No connection message?
Press Ctrl+9 and select DBeaver Sample Database (SQLite).

SIMPLER One shortcut, Ctrl+Enter, runs the statement under your cursor. That is most of what you need.

PART 1

Databases, SQL and DBeaver

What we are working with

RELATIONAL DATABASES

Tables, rows, columns and keys

CustomerId	FirstName	LastName	Country	SupportRepId
1	Luís	Gonçalves	Brazil	3
2	Leonie	Köhler	Germany	5
3	François	Tremblay	Canada	3
4	Bjørn	Hansen	Norway	4

The Customer table (first 4 of 59 rows)

Table

A set of rows that share the same columns.

Row and column

A row is one record; a column is one attribute with a type.

Primary key

Uniquely identifies each row: CustomerId.

Foreign key

Points to a row in another table: SupportRepId → Employee.

SIMPLER Store each fact once, then link it with keys. That is why we will need JOINs later.

WHAT IS SQL?

One language, many databases

Declarative

You say what you want, not how to get it. The database plans the work.

Standardized

ANSI/ISO SQL since 1986. The basics carry across products.

Everywhere

MySQL, PostgreSQL, SQL Server, Oracle, SQLite, MariaDB, DuckDB…

QUERY

`SELECT`

Read data (most of today)

CHANGE

`INSERT` · `UPDATE` · `DELETE`

Add, modify and remove rows

DEFINE

`CREATE` · `ALTER` · `DROP`

Build tables (another day)

SIMPLER — Learn SQL once and you can query almost any database you meet, from a Raspberry Pi to the cloud.

THE SAMPLE DATA

Meet Chinook: a digital music store

Table	Rows	What it holds
Artist	275	Performers
Album	347	Albums by artist
Track	3,503	Songs and videos
Genre	25	Rock, Jazz, Latin…
Customer	59	Buyers in 24 countries
Invoice	412	Purchases
InvoiceLine	2,240	Tracks on each invoice
Employee	8	Staff and support reps

Familiar subject

Everyone knows what an album, a track and a purchase are.

Small but real

Big enough for interesting questions, small enough to run instantly.

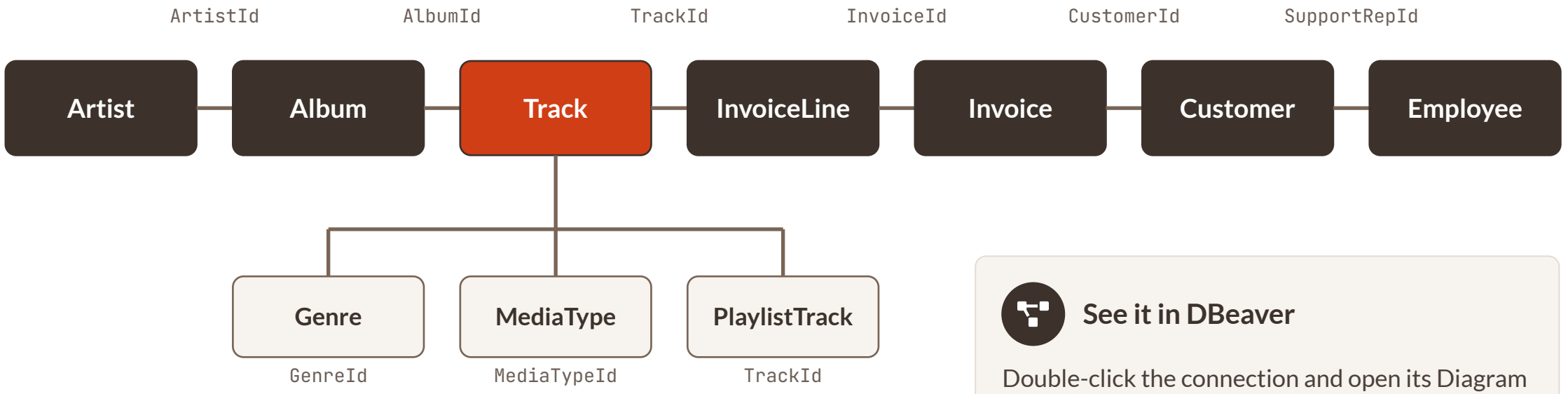
Available everywhere

Chinook also ships for MySQL, PostgreSQL, SQL Server and Oracle.

SIMPLER Real-looking data makes queries meaningful: “who are our best customers?” beats “SELECT * FROM t1”.

HOW THE TABLES CONNECT

Follow the keys from artist to customer



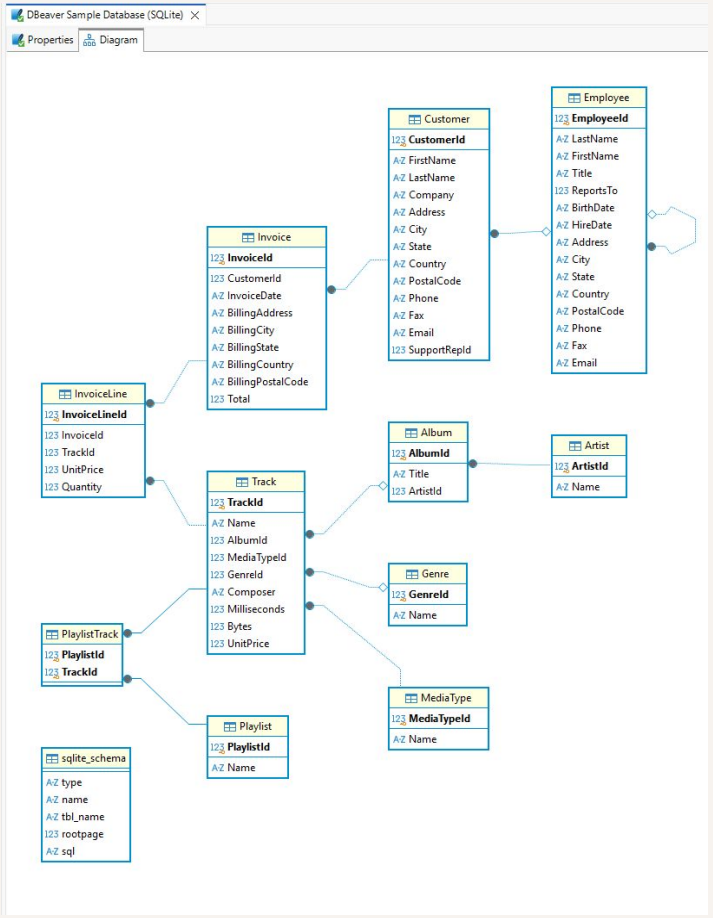
See it in DBeaver

Double-click the connection and open its Diagram tab (tables have an ER Diagram tab). Lines are foreign keys.

SIMPLER DBeaver draws this map for you, for any database you connect to.

ENTITY RELATIONSHIP MAP

How the data connects together



Boxes are tables

Each lists its columns. The bold column at the top is the primary key.

Lines are foreign keys

The filled dot marks the table holding the key: Invoice.CustomerId → Customer.

Hollow diamond = optional

The key may be NULL, e.g. a customer with no support rep.

Employee links to itself

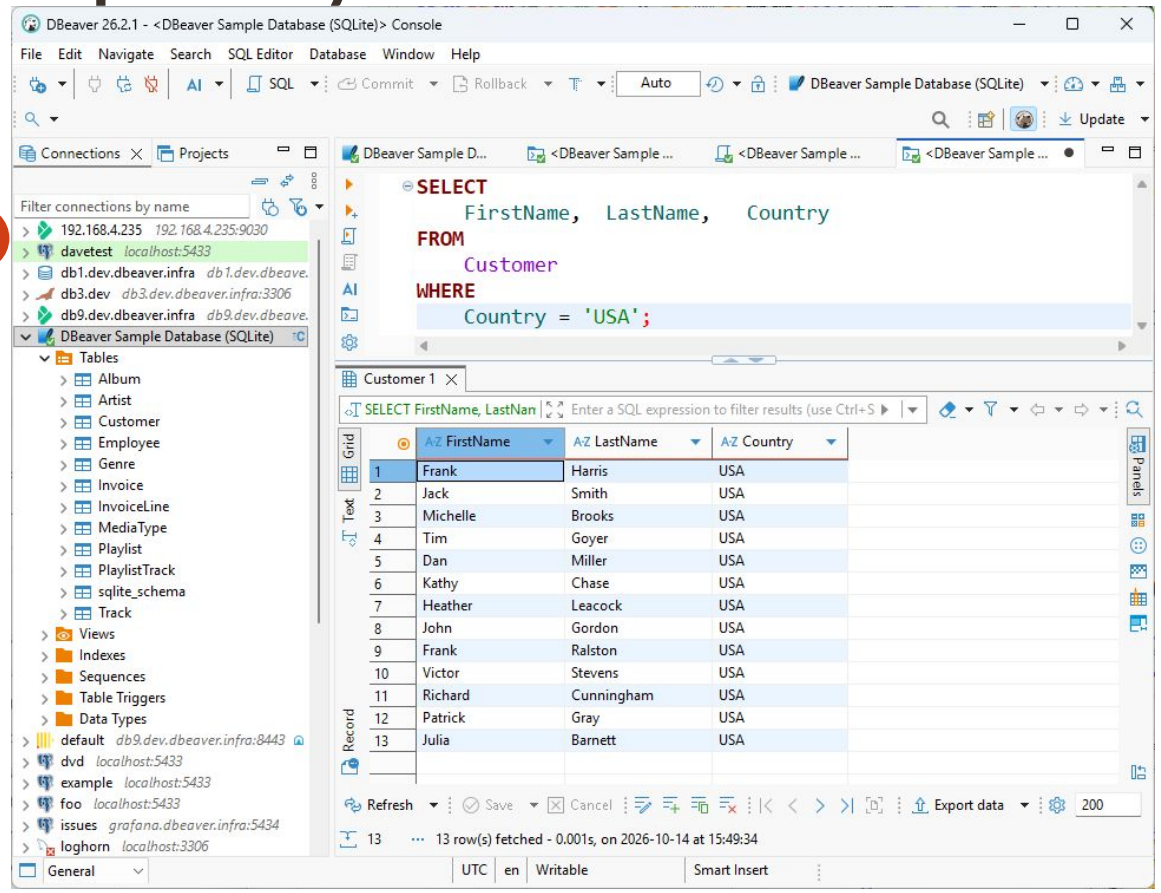
ReportsTo holds the EmployeeId of each person's manager.

sqlite_schema

SQLite's own list of tables. Not part of the music store; ignore it.

DBEAVER TOUR

Three panels you will use all afternoon



1

Database Navigator

Connections, schemas, tables. Double-click a table for Properties, Data and ER Diagram tabs.

2

SQL Editor

Write and run SQL. Autocomplete and syntax highlighting built in.

3

Results panel

Sort, filter and export what comes back, without writing more SQL.

SIMPLER Browse, write and review in one window; no switching between a terminal and a spreadsheet.

SHORTCUTS

Keys worth learning today

Keys (Linux / Windows)	What it does
`Ctrl+Enter`	Run the statement under the cursor
`Alt+X`	Run the whole script
`Ctrl+]`	New SQL script for this connection
`Ctrl+Space`	Autocomplete tables, columns, keywords
`Ctrl+Shift+F`	Format (pretty-print) the SQL
`Ctrl+/`	Comment or uncomment the line
`Ctrl+Shift+E`	Show the execution plan

On macOS

Use Cmd instead of Ctrl for most of these.

Forgot one? Right-click in the editor: every action shows its shortcut next to it.

SIMPLER Ctrl+Enter and Ctrl+Space alone will save you hundreds of mouse clicks this afternoon.

EXERCISE 0 · 1 MINUTE

Look around before we type

Task 1

Double-click the Track table and open its Data tab. How many columns does it have?

Task 2

Double-click the connection and open its Diagram tab. Find the line between Customer and Employee.

Task 3

Open a new SQL editor (Ctrl+]) and keep it open for Part 2.

Answer: Track has 9 columns (TrackId, Name, AlbumId, MediaTypeId, GenreId, Composer, Milliseconds, Bytes, UnitPrice).

SIMPLER The Data tab is a spreadsheet-like view of any table; you can look before you write a query.

PART 2

SELECT: reading data

Columns, aliases, DISTINCT, ORDER BY and LIMIT

THE SHAPE OF A QUERY

Clauses always come in this order

`SELECT`	which columns	Part 2
`FROM`	which table(s)	Parts 2 & 7
`WHERE`	which rows	Part 3
`GROUP BY`	how to bucket rows	Part 6
`HAVING`	which buckets	Part 6
`ORDER BY`	what order	Part 2
`LIMIT`	how many	Part 2

ALL TOGETHER

```
-- Only SELECT and FROM
-- are required
SELECT Name, Milliseconds
FROM Track
WHERE GenreId = 1
ORDER BY Milliseconds DESC
LIMIT 5;
```

SIMPLER Memorize the order once and SQL error messages about “syntax near …” mostly disappear.

SELECT BASICS

Everything, or just what you need

```
-- All columns
SELECT * FROM Employee;
```

```
-- Just the columns you need
SELECT FirstName, LastName, Title
FROM Employee;
```

Statements end with a semicolon. Keywords are not case-sensitive, but UPPERCASE keywords make queries easier to read.

FirstName	LastName	Title
Andrew	Adams	General Manager
Nancy	Edwards	Sales Manager
Jane	Peacock	Sales Support Agent
Margaret	Park	Sales Support Agent
Steve	Johnson	Sales Support Agent

First 5 of 8 rows

SIMPLER Name your columns: results are smaller, clearer and do not change when the table does.

ALIASES AND EXPRESSIONS

Rename and calculate as you select

```sql
SELECT FirstName || ' ' || LastName AS FullName,
       Email
FROM Customer
LIMIT 3;
```

FullName	Email
Luís Gonçalves	luisg@embraer.com.br
Leonie Köhler	leonekohler@surfeu.de
François Tremblay	ftremblay@gmail.com

```sql
SELECT Name,
       Milliseconds / 1000
         AS Seconds
FROM Track;
```

Dialect check

Joining text: || in SQLite, PostgreSQL, Oracle. CONCAT() in MySQL. + or CONCAT() in SQL Server.

SIMPLER Good aliases turn a result set into something you can hand straight to a colleague.

DISTINCT

Remove duplicate rows

```sql
SELECT DISTINCT Country
FROM Customer
ORDER BY Country;
```

```sql
-- How many countries? 24
SELECT COUNT(DISTINCT Country) FROM Customer;
```

Country
Argentina
Australia
Austria
Belgium
Brazil
Canada

First 6 of 24 rows

Watch out

DISTINCT FirstName, Country removes rows only when both columns match.

SIMPLER DISTINCT answers “what values exist?” in one line, with no spreadsheet de-duplicating.

ORDER BY

Sort the results

```sql
-- Longest tracks first
SELECT Name, Milliseconds
FROM Track
ORDER BY Milliseconds DESC
LIMIT 3;
```

Name	Milliseconds
Occupation / Precipice	5286953
Through a Looking Glass	5088838
Greetings from Earth, Pt. 1	2960293

```sql
-- Sort by country, then last name
SELECT LastName, FirstName, Country
FROM Customer
ORDER BY Country, LastName;
```

LastName	FirstName	Country
Gutiérrez	Diego	Argentina
Taylor	Mark	Australia
Gruber	Astrid	Austria

SIMPLER If the order matters, say so with ORDER BY. Never rely on the order rows happen to come back in.

LIMIT

Just the first few rows, in any dialect

Database	Top 5 longest tracks
SQLite, MySQL, PostgreSQL	… ORDER BY Milliseconds DESC LIMIT 5
PostgreSQL, Oracle, Db2	… FETCH FIRST 5 ROWS ONLY
SQL Server	SELECT TOP 5 Name … ORDER BY …

```sql
-- Rows 11-20: page 2 of a list
SELECT Name FROM Track ORDER BY Name LIMIT 10 OFFSET 10;
```

DBeaver has your back

The results panel fetches rows in pages, so SELECT * on a huge table will not freeze your laptop.

Scroll to the bottom of the grid to fetch more.

SIMPLER Use LIMIT while exploring: you see the shape of the data in milliseconds instead of waiting for every row.

LET DBEAVER HELP

Features that make SELECT easier

Autocomplete

Ctrl+Space suggests tables, columns and keywords as you type. No more typos in column names.

Format SQL

Ctrl+Shift+F turns a one-line mess into readable, indented SQL.

Sort and filter results

Click a column header to sort; type a condition in the filter box above the grid.

Export

Right-click the results › Export Data: CSV, Excel, JSON, SQL INSERTs and more.

SIMPLER Let the tool remember column names and formatting; you concentrate on the question you are asking.

EXERCISE 1 · 7 MINUTES

Your turn: SELECT

1 · Employees

Show every employee's first name, last name and hire date.

2 · Cities

List each billing city on the Invoice table once, sorted A to Z.

3 · Short tracks

Show the 5 shortest tracks: name and length in milliseconds.

Hint Use Ctrl+Space after FROM Invoice to see its columns. Task 2 needs DISTINCT and ORDER BY; task 3 needs ORDER BY and LIMIT.

SIMPLER Write it, run it with Ctrl+Enter, read the result. Fix and repeat: that loop is how everyone learns SQL.

ANSWERS

Exercise 1

TASK 1 · 8 ROWS

```sql
SELECT FirstName,
       LastName,
       HireDate
FROM Employee;
```

Andrew Adams is first, hired 2002-08-14.

TASK 2 · 53 ROWS

```sql
SELECT DISTINCT
       BillingCity
FROM Invoice
ORDER BY BillingCity;
```

Amsterdam, Bangalore, Berlin…

TASK 3 · 5 ROWS

```sql
SELECT Name,
       Milliseconds
FROM Track
ORDER BY Milliseconds
LIMIT 5;
```

Shortest: "É Uma Partida De Futebol", 1,071 ms.

SIMPLER Different queries can return the same answer; if your rows match, you got it right.

PART 3

WHERE: finding the right rows

Comparisons, AND/OR, IN, BETWEEN, LIKE and NULL

COMPARISONS

Keep only the rows that match

```
SELECT FirstName, LastName, Country
FROM Customer
WHERE Country = 'USA';
```

FirstName	LastName	Country
Frank	Harris	USA
Jack	Smith	USA
Michelle	Brooks	USA

First 3 of 13 rows

Operator	Meaning
=	equal
<> or ≠	not equal
< ⩽	less than (or equal)
> ⩾	greater than (or equal)

Text goes in 'single quotes'. Numbers do not: WHERE UnitPrice > 0.99

SIMPLER Filter in the database, not in your head: let WHERE throw away the rows you do not need.

AND, OR AND PARENTHESES

Combine conditions, and say what you mean

SURPRISE

```sql
-- Means: Brazil OR (Canada AND company)
SELECT FirstName, Country, Company
FROM Customer
WHERE Country = 'Brazil' OR Country = 'Canada'
  AND Company IS NOT NULL;    -- 7 rows
```

WHAT YOU MEANT

```sql
-- Means: (Brazil OR Canada) AND company
SELECT FirstName, Country, Company
FROM Customer
WHERE (Country = 'Brazil' OR Country = 'Canada')
  AND Company IS NOT NULL;    -- 6 rows
```

AND happens before OR

Like × before + in arithmetic. It silently changes your answer.

Always add parentheses

Whenever you mix AND and OR. Future you will thank you.

SIMPLER

Parentheses cost two keystrokes and remove all doubt about what the query means.

IN AND BETWEEN

Shorter ways to write common filters

```sql
-- A list of values
SELECT FirstName, LastName, Country
FROM Customer
WHERE Country IN ('Canada', 'Brazil',
'France');
```

18 rows. Same as three conditions joined with OR.

```sql
-- A range, inclusive at both ends
SELECT Name, Milliseconds
FROM Track
WHERE Milliseconds BETWEEN 300000 AND 360000;
```

446 rows: tracks from 5 to 6 minutes long.

NOT IN, NOT BETWEEN

Put NOT in front to flip either one.

Works on dates and text too

WHERE InvoiceDate BETWEEN '2010-01-01' AND '2010-12-31'

SIMPLER IN and BETWEEN read almost like English, so queries are easier to check at a glance.

LIKE

Pattern matching with wildcards

```sql
SELECT Name FROM Track
WHERE Name LIKE '%love%';    -- 114 rows
```

```sql
WHERE Name LIKE 'Love%'      -- starts with: 27 rows
```

```sql
WHERE FirstName LIKE '_ack'  -- Jack
```

Pattern	Matches
`'%love%'`	`love anywhere`
`'Love%'`	`starts with Love`
`'%love'`	`ends with love`
`'_ack'`	`Jack, but not Black`

Case rules differ: SQLite and MySQL usually ignore case; PostgreSQL does not (use ILIKE).

SIMPLER LIKE finds “something like this” when you only remember part of a name.

NULL

Missing values need IS NULL

WORKS

```
-- Right: 49 rows
SELECT FirstName, LastName FROM Customer
WHERE Company IS NULL;
```

NEVER MATCHES

```
-- Wrong: always 0 rows
SELECT FirstName, LastName FROM Customer
WHERE Company = NULL;
```

NULL = unknown

Not zero, not an empty string.

IS NULL / IS NOT NULL

The only reliable test.

[NULL] in DBeaver

The grid marks NULLs so you can tell them from blanks.

SIMPLER

Remember one rule, “use IS NULL”, and a whole class of puzzling empty results goes away.

EXERCISE 2 · 8 MINUTES

Your turn: WHERE

1 · Germany

Find all customers who live in Germany. Show name and city.

2 · Long tracks

How many tracks are longer than 5 minutes? (Milliseconds)

3 · Mystery composers

Find the tracks with no composer listed. How many are there?

Hint 5 minutes = 300,000 ms. To count rows, use SELECT COUNT(*) instead of listing columns (we cover COUNT properly in Part 6).

SIMPLER Write it, run it with Ctrl+Enter, read the result. Fix and repeat: that loop is how everyone learns SQL.

ANSWERS

Exercise 2

TASK 1 · 4 ROWS

```sql
SELECT FirstName,
       LastName, City
FROM Customer
WHERE Country =
      'Germany';
```

Stuttgart, Berlin (×2), Frankfurt.

TASK 2 · 1,069

```sql
SELECT COUNT(*)
FROM Track
WHERE Milliseconds
      > 300000;
```

1,069 tracks run longer than 5 minutes.

TASK 3 · 978

```sql
SELECT COUNT(*)
FROM Track
WHERE Composer
      IS NULL;
```

Did anyone try = NULL and get 0?

SIMPLER If your count differs, check for = NULL, a missing zero, or > versus >=.

PART 4

INSERT, UPDATE and DELETE

Changing data, safely

SAFETY NET FIRST

Transactions: try it, then keep it or undo it

Auto-commit

Default. Every statement is saved the moment it runs. Fast, but no undo.

Manual commit

Changes wait until you click Commit, or vanish when you click Rollback.

In DBeaver

Switch modes with the transaction button on the main toolbar. Commit and Rollback sit right next to it.

```
BEGIN;                                  -- start a transaction (SQL way)
UPDATE Employee SET Title = 'Oops' WHERE EmployeeId = 1;
ROLLBACK;                               -- never happened
```

SIMPLER Manual commit turns every change into a draft you can throw away. Use it whenever you touch real data.

INSERT

Add a new row

ADD

```
INSERT INTO Genre (Name)
VALUES ('Texas Swing');
```

CHECK

```
SELECT * FROM Genre
ORDER BY GenreId DESC
LIMIT 3;
```

GenreId	Name
26	Texas Swing
25	Opera
24	Classical

Let the database pick the key

Leave GenreId out; it is assigned for you.

Several rows at once: VALUES (…), (…), (…);

SIMPLER Always name the columns in INSERT; it keeps working when someone adds a column to the table.

UPDATE

Change existing rows: WHERE is not optional

CHANGE ONE ROW

```
UPDATE Genre
SET Name = 'Western Swing'
WHERE Name = 'Texas Swing';
```

DANGER: NO WHERE

```
UPDATE Genre
SET Name = 'Western Swing';    -- every genre!
```

1 row updated

DBeaver reports how many rows changed. Expect 1 here.

25 rows updated?

If the number surprises you, Rollback before you Commit.

Prefer the key

WHERE GenreId = 26 is more precise than matching on a name.

SIMPLER Read the “rows updated” count every time. It is the cheapest bug detector you will ever get.

DELETE

Remove rows, the careful way

1 Look first

Run a SELECT with the WHERE you plan to use.

```
SELECT * FROM Genre
WHERE Name = 'Western Swing';
```

2 Swap the verb

Replace SELECT * with DELETE. Keep the WHERE.

```
DELETE FROM Genre
WHERE Name = 'Western Swing';
```

3 Then decide

Happy with “1 row”? Commit. Otherwise roll back.

```
-- Manual-commit mode:
COMMIT;   -- or ROLLBACK;
```

Some databases refuse to delete rows that other rows still point to (a foreign key). SQLite only checks when foreign keys are switched on.

SIMPLER SELECT first, then DELETE: you see exactly what will disappear before it disappears.

SAFETY NETS

Ways to avoid a very bad afternoon

Manual commit

Nothing is permanent until you say so.

DBeaver warnings

It can ask for confirmation before UPDATE or DELETE with no WHERE.

Connection types

Mark a connection as Production for extra confirmations and a coloured editor.

Backups

Test restores, not just backups. Today: just recreate the sample database.

SIMPLER Let the tool be paranoid for you: a confirmation dialog is much cheaper than a restore.

EXERCISE 3 · 6 MINUTES

Your turn: a customer's life story

1 · INSERT

Add customer Tux Penguin, email tux@example.com, country USA, support rep 3.

2 · UPDATE

Tux asked for a new rep. Change Tux's SupportRepId to 4.

3 · DELETE

Tux has moved on. Remove the row (look before you delete!).

Hint FirstName, LastName and Email are required (NOT NULL). Find Tux again with WHERE Email = 'tux@example.com'.

SIMPLER Write it, run it with Ctrl+Enter, read the result. Fix and repeat: that loop is how everyone learns SQL.

ANSWERS

Exercise 3

TASK 1 · 1 ROW

```
INSERT INTO Customer
 (FirstName, LastName,
  Email, Country,
  SupportRepId)
VALUES ('Tux', 'Penguin',
 'tux@example.com',
 'USA', 3);
```

Tux gets CustomerId 60 automatically.

TASK 2 · 1 ROW

```
UPDATE Customer
SET SupportRepId = 4
WHERE Email =
  'tux@example.com';
```

Check: SELECT … WHERE Email = …

TASK 3 · 1 ROW

```
SELECT * FROM Customer
WHERE Email =
  'tux@example.com';

DELETE FROM Customer
WHERE Email =
  'tux@example.com';
```

Then COMMIT (or ROLLBACK to undo it all).

SIMPLER Create, read, update, delete: you now know all four verbs of everyday SQL.

BREAK

Ten-minute break

Stretch, refill, and back for functions, grouping and JOINs

PART 5

Built-in functions

Text, numbers, dates and missing values

TEXT FUNCTIONS

Clean up and reshape text

```
SELECT UPPER(LastName)  AS Upper,
       LOWER(Email)     AS Email,
       LENGTH(LastName) AS Len,
       SUBSTR(Phone, 1, 3) AS Prefix
FROM Customer
LIMIT 3;
```

Upper	Email	Len	Prefix
GONçALVES	luisg@embraer.com.br	9	+55
KöHLER	leonekohler@surfeu.de	6	+49
TREMBLAY	ftremblay@gmail.com	8	+1

Function	Does
UPPER / LOWER	change case
LENGTH	count characters
SUBSTR(s, start, n)	cut out a piece
REPLACE(s, a, b)	swap text
TRIM	strip spaces

SQLite's UPPER only changes A–Z: see GONçALVES.

SIMPLER Fix formatting in the query and every report, export and app gets clean data for free.

NUMBERS

ROUND, and the integer-division trap

```
SELECT Name,
       ROUND(Milliseconds / 60000.0, 2)
         AS Minutes
FROM Track
ORDER BY Minutes DESC
LIMIT 3;
```

Name	Minutes
Occupation / Precipice	88.12
Through a Looking Glass	84.81
Greetings from Earth, Pt. 1	49.34

TRY IT

```
SELECT 7 / 2,      -- 3   (!)
       7 / 2.0,    -- 3.5
       ROUND(3.14159, 2),  -- 3.14
       ABS(-5);    -- 5
```



Add .0 to divide

60000.0 instead of 60000 keeps the decimals.

SIMPLER One “.0” is the difference between 88.12 minutes and 88; know the rule and save an hour of debugging.

DATES

Dates are where dialects differ most

SQLITE

```sql
SELECT InvoiceDate,
       DATE(InvoiceDate) AS Day,
       strftime('%Y', InvoiceDate) AS Yr
FROM Invoice
LIMIT 3;
```

```sql
SELECT DATE('now');              -- today
SELECT MIN(InvoiceDate), MAX(InvoiceDate) FROM Invoice;
```

Database	Year of a date
SQLite	`strftime('%Y', d)`
MySQL	`YEAR(d)`
PostgreSQL	`EXTRACT(YEAR FROM d)`
SQL Server	`YEAR(d)`
Oracle	`EXTRACT(YEAR FROM d)`

SIMPLER When dates get tricky, look up your database's date functions page once and bookmark it.

COALESCE AND CASE

Fill in blanks and make decisions

COALESCE

```
SELECT FirstName,
       COALESCE(Company,
                '(none)') AS Company
FROM Customer;
```

COALESCE(a, b, c)

First value that is not NULL. Great for display.

CASE

```
SELECT Name,
  CASE
    WHEN Milliseconds < 180000 THEN 'short'
    WHEN Milliseconds < 360000 THEN 'medium'
    ELSE 'long'
  END AS Size
FROM Track;
```

CASE checks each WHEN in order and stops at the first match.

SIMPLER

CASE puts business rules (“short”, “VIP”, “overdue”) right in the query where everyone can see them.

EXERCISE 4 · 4 MINUTES

Your turn: functions

1 · Name tags

List employees as “ADAMS, Andrew”: last name in capitals, comma, first name. Sort by last name.

2 · Megabytes

Show each track’s size in MB (Bytes / 1048576), rounded to 1 decimal, largest first.

3 · Hire year

Show each employee’s name and the year they were hired.

Hint Remember || to join text, ROUND(x, 1), and the .0 trick. In SQLite the year is strftime('%Y', HireDate).

SIMPLER Write it, run it with Ctrl+Enter, read the result. Fix and repeat: that loop is how everyone learns SQL.

ANSWERS

Exercise 4

TASK 1

```
SELECT UPPER(LastName)
       || ', ' ||
       FirstName AS Name
FROM Employee
ORDER BY LastName;
```

ADAMS, Andrew · CALLAHAN, Laura · …

TASK 2

```
SELECT Name,
  ROUND(Bytes
   / 1048576.0, 1)
   AS MB
FROM Track
ORDER BY Bytes DESC;
```

Largest: Through a Looking Glass, 1010.5 MB.

TASK 3

```
SELECT FirstName,
       LastName,
  strftime('%Y',
    HireDate) AS Yr
FROM Employee;
```

Hire years in the sample run from 2002 to 2004.

SIMPLER Functions turn raw columns into the answer people actually asked for.

PART 6

Grouping and aggregates

COUNT, SUM, AVG, MIN, MAX, GROUP BY and HAVING

AGGREGATE FUNCTIONS

Many rows in, one answer out

```sql
SELECT COUNT(*)   AS Invoices,
       SUM(Total) AS Revenue,
       ROUND(AVG(Total), 2) AS AvgSale,
       MIN(Total) AS Smallest,
       MAX(Total) AS Largest
FROM Invoice;
```

Invoices	Revenue	AvgSale	Smallest	Largest
412	2328.6	5.65	0.99	25.86

COUNT how many rows

SUM add them up

AVG the mean

MIN / MAX smallest / largest

COUNT(*) counts rows. COUNT(Company) skips NULLs: 10, not 59.

SIMPLER One query replaces exporting to a spreadsheet and writing =SUM() by hand.

GROUP BY

One summary row per group

```sql
SELECT Country,
       COUNT(*) AS Customers
FROM Customer
GROUP BY Country
ORDER BY Customers DESC;
```

Rule of thumb

Every column in SELECT is either in GROUP BY or inside an aggregate.

Country	Customers
USA	13
Canada	8
France	5
Brazil	5
Germany	4

First 5 of 24 groups

SIMPLER “How many per …?” questions are always GROUP BY. Spot the word “per” and you know what to write.

A COMMON ERROR

Columns must be grouped or aggregated

AMBIGUOUS

```sql
-- City is not grouped or aggregated
SELECT Country, City, COUNT(*)
FROM Customer
GROUP BY Country;
```

CLEAR

```sql
-- Group by both, or drop City
SELECT Country, City, COUNT(*)
FROM Customer
GROUP BY Country, City;
```



SQLite: runs anyway

Returns some City from each group. Which one is not defined.



PostgreSQL & co: error

“column must appear in the GROUP BY clause or be used in an aggregate function”.

SIMPLER Follow the strict rule and your queries will run, and mean the same thing, on every database.

HAVING

WHERE filters rows, HAVING filters groups

ONLY BIG GROUPS

```
SELECT Country,
       COUNT(*) AS Customers
FROM Customer
GROUP BY Country
HAVING COUNT(*) ⩾ 5;
```

USA 13 · Canada 8 · France 5 · Brazil 5

	Runs	Can use COUNT, SUM…?
WHERE	before grouping	No
HAVING	after grouping	Yes

SALES PER YEAR

```
SELECT strftime('%Y', InvoiceDate) AS Yr,
       COUNT(*) AS Invoices,
       ROUND(SUM(Total), 2) AS Sales
FROM Invoice
GROUP BY Yr
ORDER BY Yr;
```

Yr	Invoices	Sales
2009	83	449.46
2010	83	481.45
…	…	…

Invoices run 2009–2013 in the sample

SIMPLER Two filters, two jobs: WHERE picks the rows to count, HAVING picks the results to show.

EXERCISE 5 · 9 MINUTES

Your turn: GROUP BY

1 · Media types

How many tracks are there for each MediaTypeId?

2 · Top markets

Total invoice sales per BillingCountry: top 5, biggest first, rounded to 2 decimals.

3 · Big countries

Which countries have more than 3 customers?

Hint Task 1 and 3 need COUNT(*), task 2 needs SUM(Total). Filtering on a COUNT means HAVING, not WHERE.

SIMPLER Write it, run it with Ctrl+Enter, read the result. Fix and repeat: that loop is how everyone learns SQL.

ANSWERS

Exercise 5

TASK 1 · 5 ROWS

```
SELECT MediaTypeId,
       COUNT(*) AS Tracks
FROM Track
GROUP BY MediaTypeId;
```

MediaTypeId 1 (MPEG audio) has 3,034 tracks.

TASK 2 · 5 ROWS

```
SELECT BillingCountry,
  ROUND(SUM(Total), 2)
    AS Sales
FROM Invoice
GROUP BY BillingCountry
ORDER BY Sales DESC
LIMIT 5;
```

USA 523.06 · Canada 303.96 · France 195.10

TASK 3 · 5 ROWS

```
SELECT Country,
  COUNT(*) AS Customers
FROM Customer
GROUP BY Country
HAVING COUNT(*) > 3;
```

USA, Canada, France, Brazil, Germany

SIMPLER Notice task 1 shows MediaTypeId numbers, not names. Getting the names needs a JOIN: next part.

PART 7

JOINs: combining tables

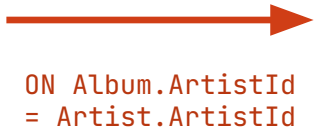
Follow the keys to answer bigger questions

WHY JOIN?

Album knows the ArtistId; Artist knows the name

AlbumId	Title	ArtistId
1	For Those About To Rock…	1
2	Balls to the Wall	2
3	Restless and Wild	2
4	Let There Be Rock	1

Album



ArtistId	Name
1	AC/DC
2	Accept
3	Aerosmith

Artist

Title	Name
For Those About To Rock We Salute You	AC/DC
Balls to the Wall	Accept

Joined result

SIMPLER JOIN puts the facts back together, so each fact can be stored in exactly one place.

INNER JOIN

Only rows that match on both sides

```
SELECT a.Title, ar.Name
FROM Album a
INNER JOIN Artist ar
        ON a.ArtistId = ar.ArtistId
WHERE ar.Name = 'AC/DC';
```

Title	Name
For Those About To Rock We Salute You	AC/DC
Let There Be Rock	AC/DC

ON says how rows match

Usually foreign key = primary key.

Aliases: a, ar

Short names for tables; prefix columns to avoid “ambiguous column” errors.

DBeaver autocomplete

Type JOIN and Ctrl+Space: it suggests the ON clause from the foreign keys.

SIMPLER With foreign keys defined, DBeaver can write most of the JOIN … ON line for you.

LEFT JOIN

Keep every row from the left table

```sql
-- Artists with no albums at all
SELECT ar.Name
FROM Artist ar
LEFT JOIN Album al
       ON ar.ArtistId = al.ArtistId
WHERE al.AlbumId IS NULL;
```

71 of 275 artists have no album of their own.

Join	Keeps
INNER JOIN	only matching rows
LEFT JOIN	all left rows, NULLs where no match
RIGHT JOIN	all right rows (rarely used)
FULL JOIN	everything from both sides

LEFT JOIN + IS NULL

The standard way to ask “which ones have none?”

SIMPLER “Customers with no orders”, “products never sold”: one LEFT JOIN pattern answers them all.

JOIN + GROUP BY

Names instead of ids

```
SELECT g.Name AS Genre,
       COUNT(*) AS Tracks
FROM Track t
JOIN Genre g ON t.GenreId = g.GenreId
GROUP BY g.Name
ORDER BY Tracks DESC
LIMIT 5;
```

Genre	Tracks
Rock	1297
Latin	579
Metal	374
Alternative & Punk	332
Jazz	130

SIMPLER JOIN for the names, GROUP BY for the counts: this pair answers most reporting questions.

FOLLOWING THE CHAIN

Top-selling artists: four tables, one query

```
SELECT ar.Name AS Artist,
       ROUND(SUM(il.UnitPrice * il.Quantity), 2)
         AS Sales
FROM Artist ar
JOIN Album al       ON ar.ArtistId = al.ArtistId
JOIN Track t        ON al.AlbumId  = t.AlbumId
JOIN InvoiceLine il ON t.TrackId   = il.TrackId
GROUP BY ar.Name
ORDER BY Sales DESC
LIMIT 5;
```

Artist	Sales
Iron Maiden	138.6
U2	105.93
Metallica	90.09
Led Zeppelin	86.13
Lost	81.59

Build it one JOIN at a time and run after each step.

SIMPLER Big queries are just small queries stacked up. Add one JOIN, run it, then add the next.

EXERCISE 6 · 5 MINUTES

Your turn: JOIN

1 · Who helps whom

List each customer's name next to their support rep's name.

2 · Rep workload

How many customers does each support rep have?

3 · Best genres

Top 5 genres by number of tracks sold (InvoiceLine).

Hint Customer.SupportRepId = Employee.EmployeeId. For task 3: InvoiceLine → Track → Genre, then SUM(Quantity).

SIMPLER Write it, run it with Ctrl+Enter, read the result. Fix and repeat: that loop is how everyone learns SQL.

ANSWERS

Exercise 6

TASK 1 · 59 ROWS

```sql
SELECT c.FirstName,
  c.LastName,
  e.FirstName AS Rep
FROM Customer c
JOIN Employee e
  ON c.SupportRepId
   = e.EmployeeId;
```

Luís Gonçalves → Jane

TASK 2 · 3 ROWS

```sql
SELECT e.FirstName,
  e.LastName,
  COUNT(*) AS Customers
FROM Customer c
JOIN Employee e
  ON c.SupportRepId
   = e.EmployeeId
GROUP BY e.EmployeeId;
```

Jane 21 · Margaret 20 · Steve 18

TASK 3 · 5 ROWS

```sql
SELECT g.Name,
  SUM(il.Quantity) AS Sold
FROM InvoiceLine il
JOIN Track t
  ON il.TrackId = t.TrackId
JOIN Genre g
  ON t.GenreId = g.GenreId
GROUP BY g.Name
ORDER BY Sold DESC
LIMIT 5;
```

Rock 835 · Latin 386 · Metal 264

SIMPLER Three tables, one question answered: that is the whole point of a relational database.

WRAP-UP

Putting it all together

One query using everything from today

CAPSTONE

Our five best customers, and who looks after them

```
SELECT c.FirstName || ' ' || c.LastName AS Customer,
       c.Country,
       ROUND(SUM(i.Total), 2) AS Spent,
       e.FirstName AS Rep
FROM Customer c
JOIN Invoice  i ON c.CustomerId   = i.CustomerId
JOIN Employee e ON c.SupportRepId = e.EmployeeId
GROUP BY c.CustomerId
HAVING SUM(i.Total) > 40
ORDER BY Spent DESC
LIMIT 5;
```

Customer	Spent	Rep
Helena Holý	49.62	Steve
Richard Cunningham	47.62	Margaret
Luis Rojas	46.62	Steve
Ladislav Kovács	45.62	Jane
Hugh O'Reilly	45.62	Jane

SELECT · || · AS · JOIN · GROUP BY · SUM · ROUND · HAVING · ORDER BY · LIMIT

SIMPLER Three hours ago this would have been a weekend with a spreadsheet. Now it is eleven lines.

TAKING IT ELSEWHERE

Same SQL, different databases

Task	SQLite	MySQL	PostgreSQL	SQL Server
First n rows	`LIMIT n`	`LIMIT n`	`LIMIT n / FETCH FIRST`	`TOP n`
Join text	`a || b`	`CONCAT(a, b)`	`a || b`	`a + b / CONCAT`
Year of a date	`strftime('%Y', d)`	`YEAR(d)`	`EXTRACT(YEAR FROM d)`	`YEAR(d)`
Case-insensitive match	`LIKE`	`LIKE`	`ILIKE`	`LIKE (collation)`
Auto-numbered key	`INTEGER PRIMARY KEY`	`AUTO_INCREMENT`	`IDENTITY / SERIAL`	`IDENTITY`

SIMPLER One DBeaver window can hold all of these connections, so you can compare dialects side by side.

BEST PRACTICES

Habits that keep you out of trouble

WHERE on every UPDATE and DELETE

And SELECT with the same WHERE first.

Use manual commit for real data

Check the row count, then Commit.

Name your columns

Avoid SELECT * outside exploring.

ORDER BY when order matters

Never trust the default order.

IS NULL, never = NULL

And remember COUNT(column) skips NULLs.

Build big queries in steps

Run after every JOIN and clause.

SIMPLER Six habits, zero cost, and you will avoid the mistakes that make people afraid of databases.

KEEP LEARNING

Where to go next

DBeaver Community

dbeaver.io/download · github.com/dbeaver/dbeaver

DBeaver documentation

dbeaver.com/docs · sample database, shortcuts, SQL editor

SQLite SQL reference

sqlite.org/lang.html

Chinook for other databases

github.com/lerocha/chinook-database

Your database's docs

postgresql.org/docs · dev.mysql.com/doc · learn.microsoft.com/sql

Today's examples

[link to your demo .sql file]

SIMPLER Keep the sample database: it is the perfect place to try anything before you run it on real data.

DBeaver

Thank you!

You can now read, filter, change, summarize and join data with SQL.

Questions?
David Stokes · David.Stokes@DBeaver.com

dbeaver.io/download
github.com/dbeaver/dbeaver